

Estrutura de Dados

Árvore Binária de Busca

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Da Montagem à Organização Automática

Na Aula 02 a árvore de zonas foi montada à mão: cada ligação nasceu de uma decisão explícita do programador. A árvore binária de busca inverte esse papel. A estrutura passa a se organizar sozinha, decidindo o lugar de cada novo elemento por comparação de chaves, sem que nenhuma ligação precise ser escrita manualmente.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

A Regra da ABB

A árvore binária de busca obedece a uma única regra, válida em qualquer subárvore: toda chave à esquerda de um nó é menor que a sua, e toda chave à direita é maior ou igual. Essa regra não vale apenas entre pais e filhos diretos — vale para toda a subárvore inteira, em qualquer profundidade.



Prof. Everaldo Graciliano Souza Ribeiro

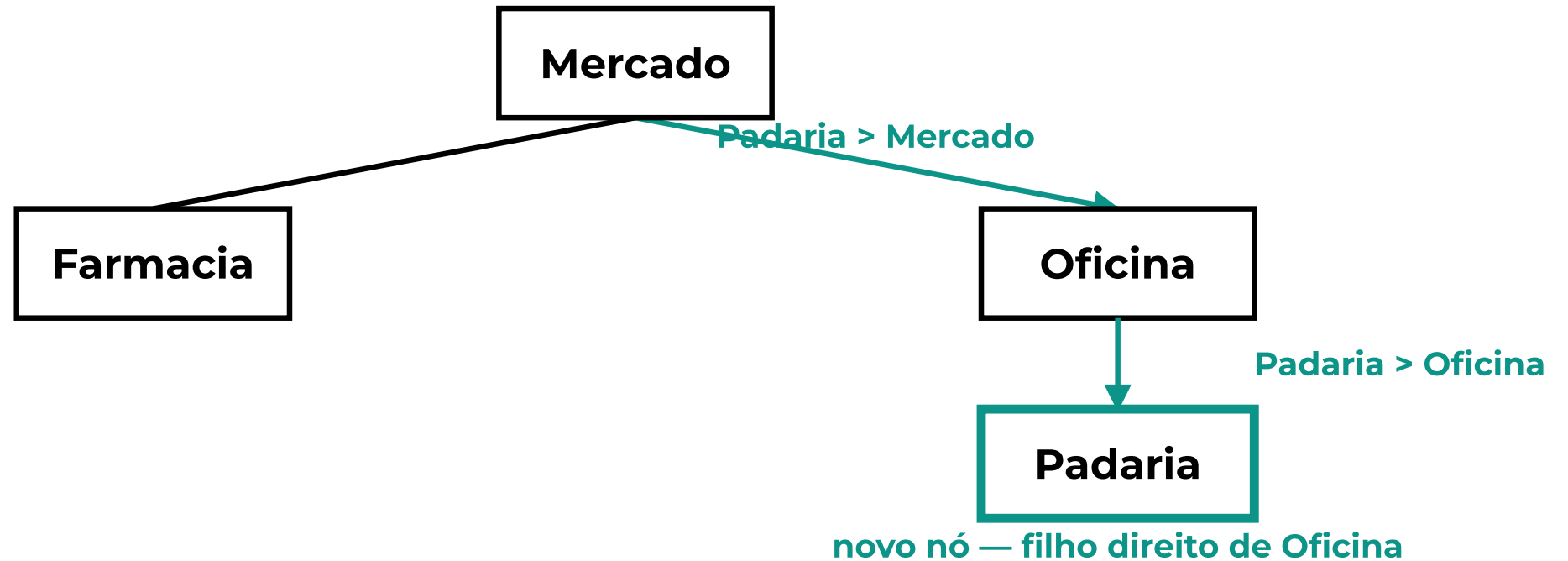
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserindo um Novo Ponto



Cada comparação decide um lado. **Duas comparações bastam** para achar onde o novo nó entra.



Inserir por Comparação

Inserir um novo elemento significa descer pela árvore comparando sua chave com a de cada nó visitado: se for menor, o caminho segue à esquerda; se for maior ou igual, segue à direita. Quando a descida encontra uma subárvore vazia, é ali que o novo nó é ligado — nunca em outro lugar.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Inserir: Descer Comparando

```
def inserir(raiz, ponto):  
    """Insere `ponto` na ABB indexada por `ponto.nome`."""  
    # Caso base: a subarvore esta vazia → o novo no nasce aqui.  
    if raiz is None:  
        return No(ponto)  
    # Compara a chave e desce para o lado correspondente.  
    if ponto.nome < raiz.valor.nome:  
        raiz.esquerda = inserir(raiz.esquerda, ponto)  
    else:  
        raiz.direita = inserir(raiz.direita, ponto)  
    return raiz
```



Buscar Sem Percorrer Tudo

A busca reaproveita a mesma lógica da inserção: compara a chave procurada com a raiz da subárvore atual e desce para um único lado. Cada comparação elimina a outra subárvore inteira do espaço de busca, sem visitar nenhum de seus nós. É essa eliminação, e não sorte, que torna a busca eficiente.



Prof. Everaldo Graciliano Souza Ribeiro

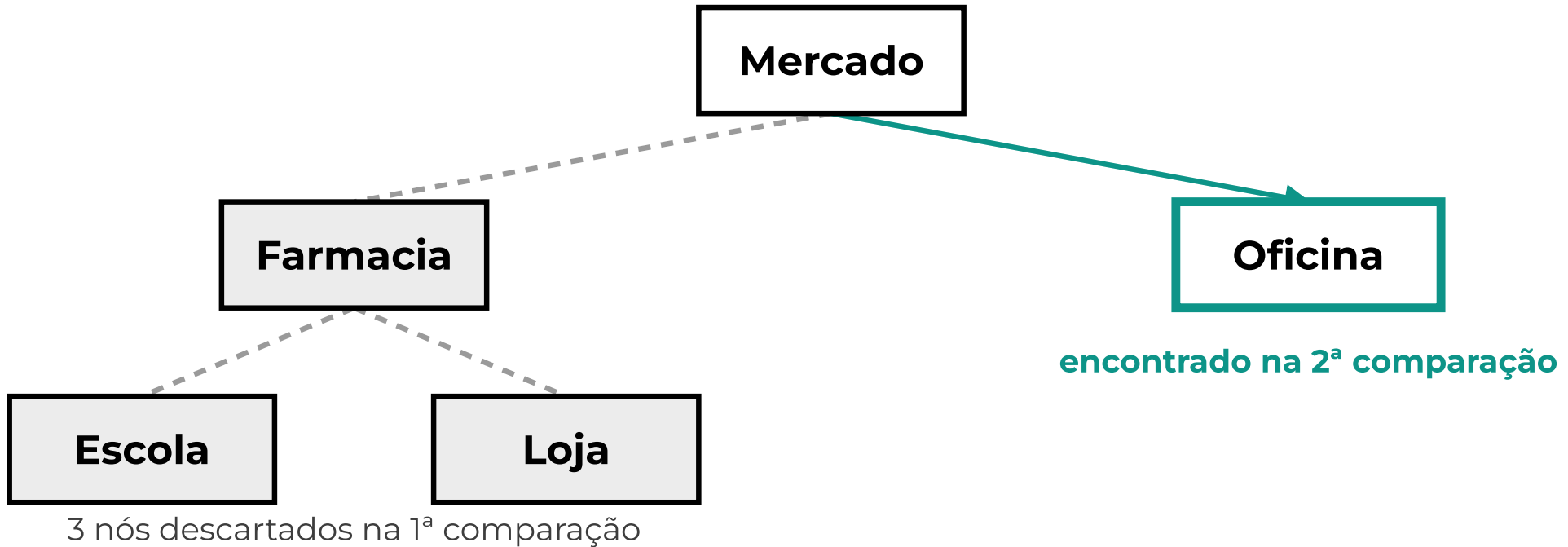
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Busca: Um Lado Descartado



Ao decidir o lado, a busca **descarta a outra subárvore inteira** sem visitá-la.



Buscar por Chave

```
def buscar(raiz, nome):  
    """Custo proporcional a altura, nao ao total de nos."""  
    if raiz is None:  
        return None  
    if nome == raiz.valor.nome:  
        return raiz.valor  
    # Uma unica comparacao descarta o outro lado inteiro.  
    if nome < raiz.valor.nome:  
        return buscar(raiz.esquerda, nome)  
    return buscar(raiz.direita, nome)
```



O Custo é a Altura, De Novo

A Aula 02 já havia mostrado que a altura, e não o número de nós, determina o custo de percorrer uma árvore da raiz até uma folha. Na ABB isso se repete para inserção e busca: cada uma delas realiza, no pior caso, tantas comparações quantos forem os níveis atravessados até o alvo.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Ordem de Inserção Decide a Forma

A forma da ABB não é escolha livre: ela é consequência direta da ordem em que os elementos chegam. Inserir chaves já ordenadas produz uma árvore degenerada, em que cada nó tem um único filho e a altura se aproxima do total de elementos. É o mesmo problema que a Aula 05 enfrenta com balanceamento.



Prof. Everaldo Graciliano Souza Ribeiro

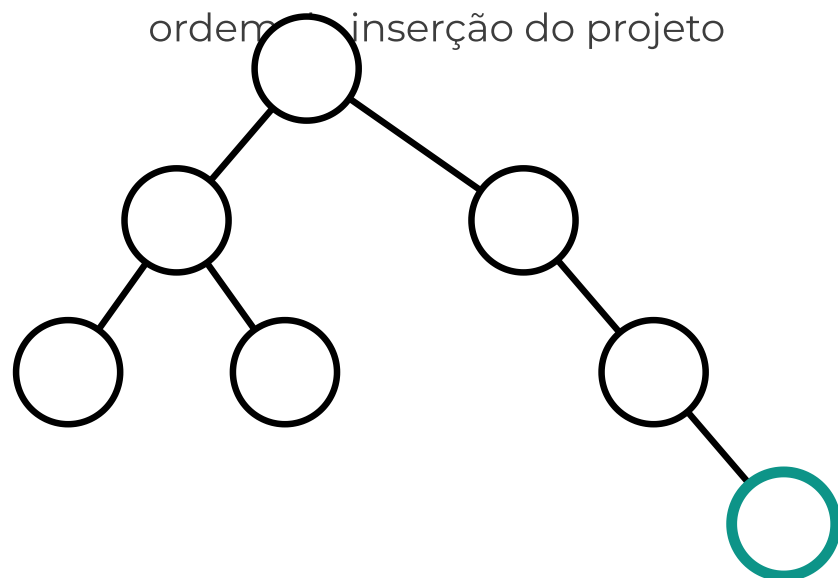
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

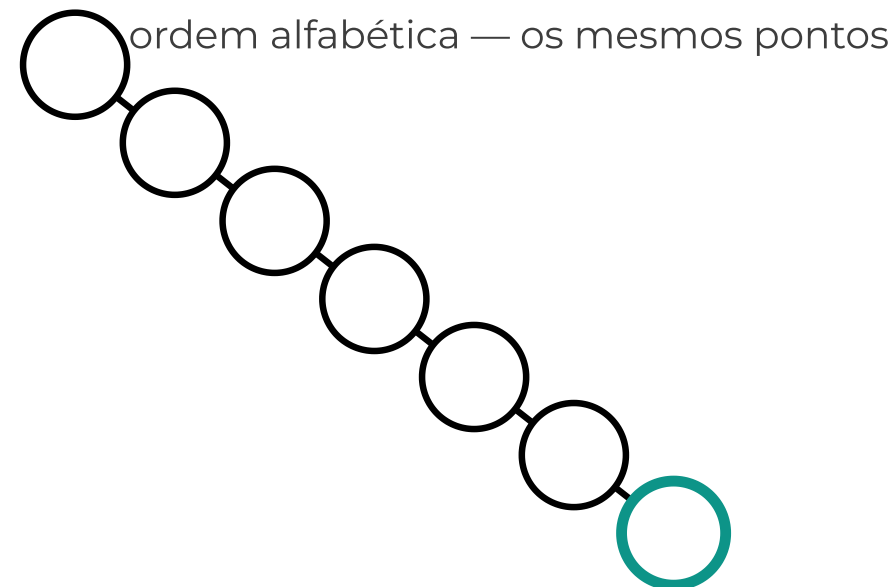
 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Mesmos Pontos, Alturas Diferentes



7 pontos — mesmas chaves do índice
altura 3 · 4 comparações no pior caso



inseridas em ordem — degenera em lista
altura 6 · 7 comparações no pior caso

A ordem de inserção decide a forma — mesmas 7 chaves, alturas 3 e 6.



Onde a ABB Entra no LogiRota

O LogiRota precisa localizar rapidamente um ponto de entrega pelo nome, entre centenas de cadastros possíveis. Uma lista percorrida do início ao fim resolveria o problema, mas a um custo proporcional ao total de pontos. A ABB indexa esses pontos pelo nome e reduz esse custo à altura da árvore.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Índice em Ação

```
indice = construir_indice(PONTOS)
print(altura(indice))                # 3

achado = buscar(indice, "Oficina Safira")
print(achado)                        # Oficina Safira (Safira)

# Mesmos pontos, ordem alfabetica: pior caso da ABB.
ordenado = sorted(PONTOS, key=lambda p: p.nome)
degenerado = construir_indice(ordenado)
print(altura(degenerado))            # 6
```



Atividades

Exercício 1

Na ABB do índice, por que Padaria Distrito termina como filho direito de Oficina Safira, e não filho direto de Mercado Barra?

Exercício 2

Quantas comparações, no máximo, uma busca faz numa ABB de altura 3? E numa de altura 6, com o mesmo número de pontos?

Exercício 3

Por que inserir os pontos já em ordem alfabética produz a pior forma possível para a ABB?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Escrever `menor(no)` e `maior(no)`, que devolvem a chave mínima e a máxima descendo sempre para um único lado.
2. Escrever `remover(raiz, nome)`, tratando o caso do nó com dois filhos — ele não pode simplesmente desaparecer.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>